

The Kalman Filter for Crypto Pairs Trading

A Self-Contained Derivation with Code Walkthrough

1 The problem

Two cryptocurrencies, say BTC and ETH, are economically linked: ETH “follows” BTC most of the time. The classical pairs-trading idea is to model the relationship as a linear regression

$$y_t = \beta x_t + \alpha + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R), \quad (1)$$

where y_t is the price of ETH and x_t is the price of BTC at time t . Estimate β, α by ordinary least squares (OLS) on a lookback window. Form the *spread*

$$s_t = y_t - \beta x_t - \alpha. \quad (2)$$

If the spread is mean-reverting (a fact you would test with an ADF or a Johansen test in real life), bet on s_t returning to zero whenever it deviates: short the spread when $s_t \gg 0$, long it when $s_t \ll 0$.

The trouble is that β in crypto is *not* constant. Liquidity shifts, narratives change, ETH decouples for weeks at a time. A static β estimated on stale data produces a “spread” that is itself trending, and the mean-reversion bet blows up.

We want β_t that updates online, adapts to regime changes, and gives us a principled estimate of how far the current spread has wandered. That is exactly what a Kalman filter does.

2 State-space formulation

Promote the regression coefficients to a hidden state that drifts:

$$\boldsymbol{\theta}_t = \begin{bmatrix} \beta_t \\ \alpha_t \end{bmatrix}, \quad \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}). \quad (3)$$

The transition matrix is the identity — coefficients follow a random walk. $\mathbf{Q} \in \mathbb{R}^{2 \times 2}$ controls how fast they drift.

The observation at time t is the price y_t , which depends linearly on the state through the regressor row

$$\mathbf{H}_t = [x_t \ 1], \quad y_t = \mathbf{H}_t \boldsymbol{\theta}_t + v_t, \quad v_t \sim \mathcal{N}(0, R). \quad (4)$$

That is, $y_t = \beta_t x_t + \alpha_t + v_t$ — equation (1) with time-varying coefficients. R is the variance of the per-step idiosyncratic spread noise.

3 Deriving the filter

The Kalman filter computes, recursively, the posterior $p(\boldsymbol{\theta}_t | y_{1:t})$. Under the linear-Gaussian assumptions above this posterior is itself Gaussian:

$$\boldsymbol{\theta}_t | y_{1:t} \sim \mathcal{N}(\hat{\boldsymbol{\theta}}_{t|t}, \mathbf{P}_{t|t}). \quad (5)$$

We need a recursion that produces $(\hat{\boldsymbol{\theta}}_{t|t}, \mathbf{P}_{t|t})$ from $(\hat{\boldsymbol{\theta}}_{t-1|t-1}, \mathbf{P}_{t-1|t-1})$ and the new observation y_t . This splits naturally into two steps.

3.1 Predict step

Given the posterior at $t - 1$, predict the state at t *before* seeing y_t . From the random-walk transition (3):

$$\hat{\boldsymbol{\theta}}_{t|t-1} = \mathbb{E}[\boldsymbol{\theta}_{t-1} + \mathbf{w}_t | y_{1:t-1}] = \hat{\boldsymbol{\theta}}_{t-1|t-1}, \quad (6)$$

$$\mathbf{P}_{t|t-1} = \text{Cov}(\boldsymbol{\theta}_{t-1} + \mathbf{w}_t | y_{1:t-1}) = \mathbf{P}_{t-1|t-1} + \mathbf{Q}. \quad (7)$$

The state mean does not change (random walk has zero drift), but the covariance *grows* by $\mathbf{Q}$ — our uncertainty about $\boldsymbol{\theta}$ has increased because it had a step to drift.

3.2 Update step

Now observe y_t . The predicted observation is

$$\hat{y}_t = \mathbf{H}_t \hat{\boldsymbol{\theta}}_{t|t-1}, \quad (8)$$

and the *innovation* (prediction error) is

$$e_t = y_t - \hat{y}_t. \quad (9)$$

Its variance combines model uncertainty and observation noise:

$$S_t = \text{Var}(e_t) = \mathbf{H}_t \mathbf{P}_{t|t-1} \mathbf{H}_t^\top + R. \quad (10)$$

The Bayesian update formula for two jointly Gaussian quantities — here $(\boldsymbol{\theta}_t, y_t) | y_{1:t-1}$ — yields linear-Gaussian conditional distributions. The posterior mean and covariance are

$$\hat{\boldsymbol{\theta}}_{t|t} = \hat{\boldsymbol{\theta}}_{t|t-1} + \mathbf{K}_t e_t, \quad (11)$$

$$\mathbf{P}_{t|t} = \mathbf{P}_{t|t-1} - \mathbf{K}_t \mathbf{H}_t \mathbf{P}_{t|t-1}, \quad (12)$$

where

$$\mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}_t^\top S_t^{-1} \quad (13)$$

is the *Kalman gain*.

Why is (13) the right correction? Some intuition:

- $\mathbf{P}_{t|t-1} \mathbf{H}_t^\top$ is the cross-covariance between the state and the predicted observation. It says how much the state should move per unit of innovation.
- Dividing by S_t normalises by total uncertainty: if observation noise R is huge (S_t large), $\mathbf{K}_t$ is small — we trust the prior over the new data point. If R is tiny, $\mathbf{K}_t$ is large — we adopt nearly the new evidence.

This is a special case of the Bayesian update for jointly Gaussian variables; under the model assumptions there is no approximation.

3.3 Initialisation: the diffuse prior

We start with no information about (β, α) . The standard trick is a *diffuse prior*: $\hat{\boldsymbol{\theta}}_{0|0} = \mathbf{0}$ and $\mathbf{P}_{0|0} = \kappa \mathbf{I}$ for some large κ (we use $\kappa = 10^4$). A large initial covariance means the very first innovation gets a huge Kalman gain, so the filter snaps onto something close to OLS within a few steps.

A consequence: the first ~ 100 estimates are unreliable. We discard them as a burn-in.

4 Tuning Q and R (where most practitioners get stuck)

The two free parameters of the filter are Q (how fast θ drifts) and R (observation noise variance). They are coupled — only their *ratio* matters in steady state — so a common parameterisation following Chan [1] is

$$Q = \frac{\delta}{1-\delta} I, \quad \delta \in (0, 1). \quad (14)$$

Small δ means slow drift (filter close to static OLS); large δ means fast drift (filter chases recent observations).

4.1 Why δ tuning is delicate for trading

For *state estimation* alone, the optimal δ is whatever makes the filter track θ_t best. For *trading*, a critical phenomenon appears: a perfectly tuned filter has *no exploitable structure left in its innovations*. This is a theorem, not an empirical observation:

$$\mathbb{E}[e_{t+k} | e_{1:t}] = 0 \quad \text{for all } k \geq 1 \quad (\text{when } Q, R \text{ are correct}). \quad (15)$$

The innovations are a martingale difference sequence. So if you tune the filter aggressively (large δ) and trade e_t , you will have *nothing to trade*.

The escape hatch: if the underlying spread is genuinely mean-reverting (as s_t in real cointegrated pairs is), use a small δ so the filter tracks θ but does *not* absorb the mean-reversion. Trade the *posterior spread*

$$s_t = y_t - \hat{\beta}_{t|t} x_t - \hat{\alpha}_{t|t}, \quad (16)$$

not the innovation. With small δ , $\hat{\theta}_{t|t}$ moves slowly enough that within the timescale of a single mean-reversion event the hedge ratio is approximately constant, and s_t inherits the mean-reversion of the underlying cointegrating relationship. We use $\delta = 10^{-6}$ for hourly crypto.

4.2 Setting R

R has units of price². For ETH/USDT around \$3,000 with hourly residuals around \$10, $R \approx 100$. We auto-calibrate by sliding a short window (30 bars), running OLS in each window, and taking the median residual variance. The window must be *short*, otherwise β -drift inflates the residuals and R comes out too large, making the filter too sluggish.

5 The trading rule

5.1 Signal

The posterior spread (16) is in dollar units and its scale is data-dependent. Standardise with a rolling z-score over a window of W bars (we use $W = 100$):

$$z_t = \frac{s_t - \mu_t^{(W)}}{\sigma_t^{(W)}}, \quad \mu_t^{(W)} = \frac{1}{W} \sum_{k=t-W+1}^t s_k, \quad \sigma_t^{(W)} = \text{std}(s_{t-W+1:t}). \quad (17)$$

The empirical z-score is more robust than the analytic $z_t = e_t / \sqrt{S_t}$ because the analytic version is sensitive to mis-specification of R and Q .

5.2 Position

Two thresholds, $z_{\text{entry}} > z_{\text{exit}} \geq 0$. The position $p_t \in \{-1, 0, +1\}$ evolves by

$$p_t = \begin{cases} +1 & \text{if } p_{t-1} = 0 \text{ and } z_t < -z_{\text{entry}}, \\ -1 & \text{if } p_{t-1} = 0 \text{ and } z_t > +z_{\text{entry}}, \\ 0 & \text{if } p_{t-1} \neq 0 \text{ and } |z_t| < z_{\text{exit}}, \\ -p_{t-1} & \text{if } p_{t-1} \cdot z_t > z_{\text{entry}} \text{ (flip)}, \\ p_{t-1} & \text{otherwise.} \end{cases} \quad (18)$$

Convention: $p_t = +1$ means *long the spread* — long one unit of y , short β_t units of x . $p_t = -1$ is the mirror image.

5.3 PnL

A position p_t chosen at time t (using only information through t) is held into $t + 1$. Its dollar PnL is realised at $t + 1$ from the price moves $t \rightarrow t + 1$:

$$\text{PnL}_{t+1} = p_t \cdot \left[(y_{t+1} - y_t) - \beta_t(x_{t+1} - x_t) \right]. \quad (19)$$

Note the use of β_t (not β_{t+1}): the hedge ratio at the moment of the trade, not the future one. Using β_{t+1} would peek into the future.

Why does this formula give the spread's change? The portfolio held into $t + 1$ is $+p_t$ units of y and $-p_t\beta_t$ units of x . Its dollar change between t and $t + 1$ is

$$p_t(y_{t+1} - y_t) + (-p_t\beta_t)(x_{t+1} - x_t) = p_t[(y_{t+1} - y_t) - \beta_t(x_{t+1} - x_t)],$$

exactly (19). This tracks the change in spread s_t provided $\beta_t \approx \beta_{t+1}$ (slow drift) and $\alpha_t \approx \alpha_{t+1}$, which is the regime the small- δ filter operates in.

5.4 Costs

We charge a fraction τ (basis points $\times 10^{-4}$) on the y -leg notional for every position change:

$$\text{Cost}_t = \tau \cdot |y_t| \cdot |p_t - p_{t-1}|. \quad (20)$$

This is a conservative round-trip approximation; a stricter version would charge per-leg notional on both y and βx .

6 Mapping the math to the code

The implementation lives in `kalman.pairs_trading.py`. The Kalman loop is:

```
for t in range(n):
    # PREDICT
    P = P + Q # P_{t/t-1} = P_{t-1/t-1} + Q

    # UPDATE
    H = np.array([x[t], 1.0]) # H_t = [x_t, 1]
    y_hat = H @ state # H_t * theta_{t/t-1}
    e = y[t] - y_hat # innovation e_t
    S = H @ P @ H + R # innovation variance S_t
    K = (P @ H) / S # Kalman gain K_t
    state = state + K * e # theta_{t/t}
    P = P - np.outer(K, H) @ P # P_{t/t}
```

Each line is a literal transcription of the equations of Section 3. The PnL line in the backtest:

```
pnl_spread[1:] = pos[:-1] * ((y[1:] - y[:-1])
                             - beta[:-1] * (x[1:] - x[:-1]))
```

is (19) vectorised. The slice `pos[:-1]` ensures position at t is paired with the move into $t + 1$, and `beta[:-1]` uses the contemporaneous hedge ratio.

7 What the figure shows

The four-panel figure produced by the script:

1. **Prices.** ETH and BTC on twin axes. They co-move but not perfectly.
2. **Estimated $\hat{\beta}_t$.** The Kalman filter’s hedge-ratio estimate over time. On synthetic data with a sinusoidal true β , the curve traces it out cleanly. On real ETH/BTC data the curve wanders between ~ 0.04 and ~ 0.075 over a few months — regime change a static OLS could not capture.
3. **Spread z-score.** z_t from (17). The horizontal bands at ± 2 are entry triggers, the band at ± 0.5 the exit.
4. **Cumulative PnL.** Equity curve net of costs. Green/red shading marks long-spread / short-spread periods.

8 Verification (what could go wrong, and what does not)

The companion test suite in `verify.py` runs 27 checks. The non-trivial ones, with what they would catch:

Check	What a failure would mean
$\hat{\beta}_t \rightarrow \beta_{\text{OLS}}$ as $\delta \rightarrow 0$	Filter equations or matrix shapes are wrong.
$\text{acf}(e_t, 1) \approx 0$	Filter is mis-specified or has a bug; innovations should be white.
$\text{Var}(z_t) \approx 1$ in steady state	S_t is computed wrong, or R is grossly miscalibrated.
P_t symmetric and PSD throughout	Numerical instability; would force switching to Joseph form.
Past PnL invariant to future price changes	Lookahead bug — the strategy is cheating.
PnL > 0 on synthetic mean-reverting data, multiple seeds	Sign error in position or PnL formula.

The most subtle bug to avoid is in PnL alignment: pairing p_t with the price move from $t - 1 \rightarrow t$ instead of $t \rightarrow t + 1$ inverts every trade. We caught exactly this during development: the original implementation used `np.diff` with an off-by-one prepend and produced a Sharpe of -17 . After fixing the alignment to (19) the strategy produced Sharpe ≈ 2 on the same data — the change of sign that follows from trading on the right side of the bar.

9 Honest caveats

- **Sensitivity to R .** Halving the auto-calibrated R flips total PnL from positive to negative on our synthetic data. The strategy is not blindly robust to filter misspecification.
- **Cointegration is not given.** This script assumes the pair is cointegrated. In practice, you should run an Engle–Granger or Johansen test on the residuals before deploying. Many

“correlated” crypto pairs are not cointegrated; their spread has a unit root and no mean to revert to.

- **Costs are optimistic.** 4 bps per turnaround on the y -leg only undercounts the true cost. Production sizing also has to handle funding rates if you use perpetuals, slippage at size, and short availability for the spot leg.
- **No execution model.** The backtest assumes you can trade at the close of each bar. Realistic execution introduces additional slippage and timing noise.
- **Overfitting risk on δ .** $\delta = 10^{-6}$ is a reasonable default but it is one number on a continuum; a serious backtest would split into in-sample / out-of-sample and walk forward.

10 Further reading

References

- [1] Ernest P. Chan. *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley, 2013. The pairs-trading-with-Kalman chapter is the canonical reference.
- [2] Andrew C. Harvey. *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press, 1989. The standard rigorous treatment.
- [3] James Durbin and Siem Jan Koopman. *Time Series Analysis by State Space Methods*. Oxford University Press, 2nd ed., 2012. Modern and complete.
- [4] Ganapathy Vidyamurthy. *Pairs Trading: Quantitative Methods and Analysis*. Wiley, 2004. The classical cointegration view, useful as background to the static case (1).